

DET-006 | LOLBins - Living Off the Land Execution

ID: DET-006

MITRE ID: T1218

Data Source: DeviceProcessEvents

Severity: Medium

Tactic: Defense Evasion

Category: Endpoint

ADX MAPPING LOGIC:

Mapping: EventType = 'LOLBin binary name', BeginLat/BeginLon = 'network beacon coords', DamageProperty = 'bytes transferred proxy'. We use toscalar() and between() operators.

OBJECTIVE: Practice toscalar() and between() — key operators for LOLBin detection thresholds. Real Sentinel: mshta.exe / certutil.exe with http/ftp/base64 in the command line.

KQL QUERY (RUNNABLE ON ADX FREE TIER)

```
let TotalEvents = toscalar(
StormEvents | where StartTime > ago(365d) | count);
let SuspiciousTypes = dynamic(["Tornado","Waterspout","Dust Devil"]);//= LOLBin binaries
StormEvents
| where StartTime > ago(365d)
| where EventType in (SuspiciousTypes)
| summarize
ExecutionCount = count(),
UniqueLocations = dcount(State),
AvgDamage = avg(DamageProperty),
MaxDamage = max(DamageProperty),
TotalInjuries = sum(InjuriesDirect),
LocationList = make_set(State, 5)
by EventType, Source
| extend
GlobalFreqPct = round(100.0 * ExecutionCount / TotalEvents, 2),
RiskScore = iff(MaxDamage > 100000, "HIGH", iff(MaxDamage > 10000,
"MEDIUM","LOW"))
| where ExecutionCount between (5 .. 500) // not too rare, not too noisy
| project EventType, Source, ExecutionCount, GlobalFreqPct, RiskScore, AvgDamage,
MaxDamage, UniqueLocations, LocationList
| order by MaxDamage desc | take 20
```

INVESTIGATION STEPS

1. Analyze the full command line for download/execution patterns.
2. Check network logs for connections to external IPs/domains.
3. Identify what file was downloaded or executed.
4. Check if the binary was running from an unusual path.

RESPONSE ACTIONS

- Block the LOLBin usage pattern via ASR rules in Defender.
- Isolate endpoint if network callback is detected.
- Hash the downloaded payload and hunt across all devices.
- Review and harden AppLocker / WDAC policies.

Output from Above KQL Query:

```
1 let TotalEvents = toscalar(
2 StormEvents | where StartTime > ago(365d) | count);
3 let SuspiciousTypes = dynamic(["Tornado","Waterspout","Dust Devil"]);
4 StormEvents
5 | where StartTime > ago(365d)
6 | where EventType in (SuspiciousTypes)
7 | summarize
8 ExecutionCount = count(),
9 UniqueLocations = dcount(State),
10 AvgDamage = avg(DamageProperty),
11 MaxDamage = max(DamageProperty),
12 TotalInjuries = sum(InjuriesDirect),
13 LocationList = make_set(State, 5)
14 by EventType, Source
15 | extend
16 GlobalFreqPct = round(100.0 * ExecutionCount / TotalEvents, 2),
17 RiskScore = iff(MaxDamage > 100000, "HIGH", iff(MaxDamage > 10000, "MEDIUM", "LOW"))
18 | where ExecutionCount between (5 .. 500) // not too rare, not too noisy
19 | project EventType, Source, ExecutionCount, GlobalFreqPct, RiskScore,
20 AvgDamage, MaxDamage, UniqueLocations, LocationList
21 | order by MaxDamage desc | take 20
```

Table 1 + Add visual Stats 2026-06-02 10:27 (UTC) Done (1.883 s)

EventType	Source	ExecutionCount	GlobalFreqPct	RiskScore	AvgDamage	MaxDamage	UniqueLocations	LocationList
-----------	--------	----------------	---------------	-----------	-----------	-----------	-----------------	--------------

No Rows To Show

KQL Detection Query in Sentinel:

DeviceProcessEvents

| where TimeGenerated > ago(1h)

| where FileName in~ (

"mshta.exe", "regsvr32.exe", "rundll32.exe",

"certutil.exe", "bitsadmin.exe", "wmic.exe",

"msiexec.exe", "installutil.exe", "cmstp.exe",

"msdt.exe", "pcalua.exe"

)

| where ProcessCommandLine has_any (

"http", "ftp", "\\", "base64",

"decode", "urlcache", "transfer"

)

| project

TimeGenerated, DeviceName, AccountName,

FileName, ProcessCommandLine,

InitiatingProcessFileName,

FolderPath

| order by TimeGenerated desc